P6 - GENERATIVE AI MINI-PROJECT

Next-Token Probability Lab

An interactive application for exploring token probability, self-attention, perplexity, sampling, model comparison, and safety in language models.

Name	Akash Mahadeo Kurane
Class	LY AIA
Enrolment No.	ADT23SOCB0094
Roll No.	06
Subject	Generative AI

PROJECT REPORT - WITH DEPLOYMENT EVIDENCE

Problem and Objective

Project title: Next-Token Probability Lab (P6). The objective is to build an educational Streamlit application that makes the internal behavior of autoregressive and masked language models inspectable. The application presents next-token probabilities, final-layer attention, perplexity, BERT-versus-GPT-2 predictions, controlled PyTorch sampling, generated visual output, and safety analysis in one interface.

Project objectives

- Calculate and visualize GPT-2 next-token distributions.
- Extract final-layer self-attention and calculate perplexity.

- Compare BERT masked-token predictions with GPT-2 causal predictions.
- Implement PyTorch sampling and Pillow text-to-image output.
- Apply safety filtering and analyze temperature-related factual errors.

Scope: This is an educational explorer, not a production moderation, search, or factuality system. Model scores are model-specific and generated text must be reviewed.

Next-Token Probability Lab | Generative AI | Roll No. 06

1. System Overview

The application is organized into a presentation layer and an inference layer. Streamlit handles layout, controls, charts, warnings, and user interaction. The models module handles model loading, tokenization, inference, sampling, safety checks, and Pillow image generation.

Layer	Implementation	Responsibility
User interface	app.py / Streamlit	Controls, sections, charts, metrics, warnings, and responsive glassmorphism styling.
Inference logic	models.py / PyTorch + Transformers	Model loading, probabilities, attention, perplexity, BERT predictions, generation, safety, and image rendering.
Visualization	Plotly, Matplotlib, Seaborn, Pillow	Interactive probability chart, attention heatmap, and generated text canvas.
Models	gpt2 and bert-base-uncased	Causal language modeling and masked language modeling.

Processing flow

Prompt → tokenizer → model logits → temperature scaling → top-k/top-p filtering → probability chart or multinomial sampling → safety filter → displayed text and image.

Design principles

- Separation of concerns between app.py and models.py.
- CPU-safe plotting by moving tensors to CPU before visualization.
- Graceful handling of empty input, missing masks, and model errors.
- Minimal third-party dependencies.

2. U1 - Probability Lab and Sliders

The U1 section accepts a prompt and evaluates the final GPT-2 logits at the last input position. Logits are divided by temperature before softmax. The top-k token IDs are decoded and displayed with normalized probabilities.

Control	Range	Effect
Temperature	0.1 to 2.0	Low values concentrate probability; high values flatten the distribution.
Top-k	1 to 100	Keeps only the k highest-scoring candidate tokens.
Top-p	0.1 to 1.0	Keeps the smallest candidate set whose cumulative probability reaches p.

Implementation evidence

The horizontal Plotly chart ranks decoded candidate tokens and annotates each bar with a percentage. Token labels preserve whitespace and punctuation as much as possible so learners can inspect the model vocabulary.

Interpretation

If one token has a high probability and alternatives have much smaller values, the model is concentrated at that position. Increasing temperature makes alternatives more competitive; it does not create new knowledge.

next_token_probabilities(model, prompt, top_k, temperature)

3. U2 - Attention and Perplexity

GPT-2 is called with attention outputs enabled. The final transformer layer is averaged across heads to form a token-by-token matrix. The matrix is detached and moved to CPU before it is sent to Seaborn for heatmap rendering.

Perplexity calculation

For a sequence, the model loss is the mean negative log-likelihood of the target tokens. Perplexity is calculated as exp(loss). Lower values indicate that the model assigned higher likelihood to the observed sequence under the same tokenizer and evaluation conditions.

Comparison with a standard LSTM baseline

An LSTM language model can report the same perplexity metric, but values should only be compared when tokenization, corpus, sequence handling, and evaluation split are equivalent. GPT-2 uses self-attention and

can model long-range relationships differently from a standard recurrent LSTM.

Interpretation caution

Attention is a useful diagnostic visualization, but a bright cell should not automatically be interpreted as a complete causal explanation of a prediction. It shows where a selected layer's head-averaged representation placed weight for the selected input.

4. U3 - BERT versus GPT-2

This section demonstrates two different language-modeling objectives. BERT receives a sentence containing the literal [MASK] token and predicts the missing token using bidirectional context. GPT-2 receives the same sentence with the mask removed and predicts the next token using only left-to-right context.

Model	Objective	Context	Output
BERT	Masked language modeling	Tokens on both sides of [MASK]	Top five replacements
GPT-2	Causal language modeling	Preceding tokens only	Top five next-token candidates

Example input

The capital of France is [MASK].

The side-by-side display makes it clear that a masked model can use right-hand context, while a causal model is designed to continue a sequence from its prefix. Scores are model probabilities, not guaranteed truth values.

5. U4 - PyTorch Sampling and Text-to-Image

The generation implementation does not use the Hugging Face pipeline. It manually performs the autoregressive loop: encode the prompt, run the model, select the last-position logits, apply temperature, apply top-k and top-p filtering, sample with torch.multinomial, append the next token, and stop at the end-of-sequence token or requested token limit.

Generation algorithm

1. Encode the prompt and move input IDs to the selected device.
2. Run GPT-2 on the current context.
3. Scale logits by temperature.
4. Remove tokens outside top-k and the nucleus top-p set.
5. Convert filtered logits to probabilities and sample with torch.multinomial.

6. Append the sampled ID and repeat.

Text-to-image output

The final generated text is drawn onto a dynamically created Pillow canvas with a dark gradient-like background, rounded panel, title, and wrapped text. The image is generated programmatically for each successful result and displayed in Streamlit.

The implementation keeps the generation loop explicit so the effect of temperature, top-k, top-p, and multinomial sampling is visible and auditable.

6. U5 - Safety Guardrails and Bias

Before generated text is displayed, it is passed through a lightweight keyword-based safety filter. The filter checks selected violent-threat and targeted-abuse patterns. When triggered, the application blocks the output and displays a warning instead of rendering the generated content.

Why a fallback filter?

The project keeps dependencies minimal. A production system would require an evaluated moderation model, multilingual coverage, adversarial testing, logging, human-review paths, and governance. The current filter is an educational guardrail, not a complete safety solution.

Temperature and factual errors

At low temperatures, generation favors high-probability continuations. This can improve consistency but may repeat a confident misconception. At high temperatures, the distribution is flatter and output is more diverse, but unsupported claims, contradictions, and hallucinations become more likely. Temperature changes sampling behavior; it does not improve factual knowledge.

Bias considerations

GPT-2 and BERT learn statistical patterns from their training data. Their predictions can reflect stereotypes, uneven representation, and historical bias. Probability charts and attention plots should be interpreted as model behavior, not objective measures of truth or social value.

7. User Interface and Visual Design

The Streamlit interface uses custom CSS to create an Apple-inspired presentation: clean typography, soft blue gradients, rounded glass-like cards, restrained borders, and responsive columns. The UI is divided into five task sections so a learner can move from token probabilities to attention, model comparison, generation, and safety analysis.

Interaction design

- Sidebar controls remain visible while the main workspace presents analysis.
- Buttons are used for expensive operations such as BERT loading and generation.
- Charts and images use the available width for readability.
- Warnings appear in context when validation or safety filtering prevents an operation.

Error handling

The application checks empty prompts, validates [MASK] for BERT, limits generation length, handles model-loading errors, and protects visualization code with user-facing exception messages. Models are cached as Streamlit resources so repeated interactions do not reload them.

8. Testing and Deployment

The source files were checked for Python syntax validity. The application is designed to run locally or on Streamlit Community Cloud with no API keys.

Test case	Expected result
Empty prompt	A validation message asks the user to enter a prompt.
BERT input without [MASK]	A warning explains that the literal [MASK] token is required.
Top-k and top-p controls	Values remain within specified ranges and sampling completes.
Safety trigger	Generated output is blocked and a warning is displayed.
CPU-only machine	Models and plots work using CPU tensors.
First launch	Models download from Hugging Face and are cached later.

Local deployment

Install dependencies with **pip install -r requirements.txt** and launch with **streamlit run app.py**. The first run requires internet access to download gpt2 and bert-base-uncased. Streamlit Community Cloud can deploy the GitHub repository using app.py as the entry point.

9. Limitations and Future Scope

Current limitations

- The safety filter is keyword-based and may miss context, coded language, or multilingual unsafe

content.

- GPT-2 and BERT are educational models and are not suitable for high-stakes factual systems.
- Perplexity is meaningful only under matched tokenization and evaluation conditions.
- Attention visualizations are diagnostic summaries, not complete explanations.
- Model downloads and CPU inference can take time on first launch.

Future improvements

- Add an evaluated safety classifier with documented thresholds.
- Add multiple prompts, downloadable charts, and experiment history.
- Add GPU-aware batching and performance metrics.
- Add formal evaluation for perplexity, factuality, and safety behavior.
- Add authentication and rate limiting before public deployment.

Conclusion

Next-Token Probability Lab combines model introspection and controlled generation in a compact educational application. It demonstrates how GPT-2 probabilities, attention, perplexity, BERT masking, PyTorch sampling, and safety reasoning fit together in an end-to-end Generative AI workflow.

References

1. Radford, A. et al. Improving Language Understanding by Generative Pre-Training. OpenAI, 2018.

2. Devlin, J. et al. BERT: Pre-training of Deep Bidirectional Transformers for Language Understanding. NAACL, 2019.

3. Wolf, T. et al. Transformers: State-of-the-Art Natural Language Processing. EMNLP, 2020.

4. PyTorch Documentation. Tensors, inference mode, top-k selection, and multinomial sampling.

5. Streamlit Documentation. Application deployment and resource caching.

6. Hugging Face Model Cards. gpt2 and bert-base-uncased.

Project repository: https://github.com/codedbyadiiii/next-token-probability-lab

Subject: Generative AI | Roll No. 06 | Project: P6